

- **M2M devices:** These are devices that are capable of replying to the request for data contained within those devices or capable of transmitting data in an autonomous manner. Examples are sensors, WPAN technologies like ZigBee or Bluetooth, LoWPAN, etc.
- **M2M area network (device domain):** This enables connectivity between M2M devices and M2M gateways. An example is a personal area network.
- **M2M gateway:** This utilises M2M capabilities to ensure M2M devices are interoperable and interconnected to the communications network. Gateways and routers are endpoints of the operator's network in scenarios where sensors and M2M devices connect to the network.
- **M2M communication networks:** These comprise communication between M2M gateways and M2M applications. Examples are xDSL, LTE, WiMAX and WLAN.
- **M2M applications:** All M2M applications are based on assets provided by the operator. Examples are IoT based smart homes, e-health, m-health, telemedicine and the Internet of Medical Things, vending machines, smart parking systems, autonomous store payments and wireless payment systems, digital control systems in factories, smart IIoT, industrial monitoring, etc.

There are various M2M open source development platforms. In this article, the primary focus is on DeviceHive, an open source M2M development platform.

Introducing DeviceHive

DeviceHive was created and launched by DataArt, a boutique software development and outsourcing company in New York city, as an open source M2M communications framework using which developers could design M2M projects. It provides powerful instruments for smart devices to communicate and

manage services. It incorporates three critical technologies that affect mobile developers and users—the cloud, the mobile and embedded. It consists of a communications layer, control software and multi-platform libraries and clients to bootstrap the development of remote sensing, remote control, monitoring and automation, smart energy, etc.

DeviceHive provides a strong foundation and building support to create or customise any IoT/M2M solution, bridging the gap between embedded development, cloud platforms, Big Data and client applications. It is a scalable, hardware and cloud agnostic microservice-based platform with device-management APIs in varied protocols, which allows end users to set up and monitor device connectivity, and perform data analytics.

Features

Listed below are the features of DeviceHive.

- **Deployment:** DeviceHive facilitates innumerable deployment options and is suitable for every organisation, whether a startup or big enterprise. It includes Docker Compose and Kubernetes deployment to facilitate the development of public, private or hybrid clouds. Various DeviceHive services are started using Docker Compose — DeviceHive Frontend service, DeviceHive backend service, DeviceHive Auth service, DeviceHive Admin Console, DeviceHive WebSocket Proxy, DeviceHive Nginx Proxy, Kafka, PostgreSQL and Hazelcast IMDG.
- **Scalability:** DeviceHive includes outstanding software design practices like a container service oriented architecture approach, managed and orchestrated by Kubernetes, which brings about scalability and availability in seconds.
- **Connectivity:** It supports connectivity with any device via the REST API, WebSockets or MQTT. It supports libraries written in varied

languages — both Android and iOS. It even supports embedded devices like ESP8266.

- **Seamless integration:** DeviceHive supports seamless integration with voice assisted services like Google, Siri and Alexa by enabling users to run customised JavaScript code.
- **Smart analytics:** It supports smart analytics using ElasticSearch, Apache Spark, Cassandra and Kafka for real-time processes. It also facilitates machine learning support.
- **Open source:** Comes under the Apache 2.0 licence for free use, and end users are supported by DataArt's IoT professionals.

Protocols, client libraries and devices supported by DeviceHive

Protocols: DeviceHive supports the REST, WebSocket API and MQTT protocols. In addition to this, for all RESTful services, it provides the Swagger API tool to test installation and other capabilities.

Client/device libraries:

DeviceHive supports numerous device libraries — the .NET framework, .NET Micro Framework, C++, Python and C (microcontrollers).

Client libraries: These include the .NET framework, iOS and the Web/JavaScript.

Device support: DeviceHive supports any device with Python, Node.js or Java based Linux boards via the DeviceHive Client library. It also supports the ESP8266 chip with a simple API to handle all types of sensors for things like temperature (DS18B20, LM75A/LM75B/LM75C, DHT11 and DHT22) and pressure (BMP180, BMP280, etc).

DeviceHive is a microservice based system, with high scalability and availability. Figure 1 highlights the microservices architecture of DeviceHive.

Components of DeviceHive

The following are the components of DeviceHive.